Vulnerability Assessment & Penetration Testing Report

1. Executive Summary

The objective of this assessment was to identify and exploit common web vulnerabilities using the **OWASP Juice Shop**, a specialized, deliberately vulnerable web application. This simulation allows for the practical application of penetration testing techniques, including **SQL Injection**, **XSS**, and **Logic Flaws**, in a safe and ethical environment.

---

2. Methodology & Target Environment

- **Primary Target:** OWASP Juice Shop (Simulation Environment).
- **Testing Tools: OWASP ZAP** (Automated Scanning) and Manual Browser-based exploitation.
- **Standard:** The audit was conducted following the **OWASP Top 10** vulnerabilities framework.

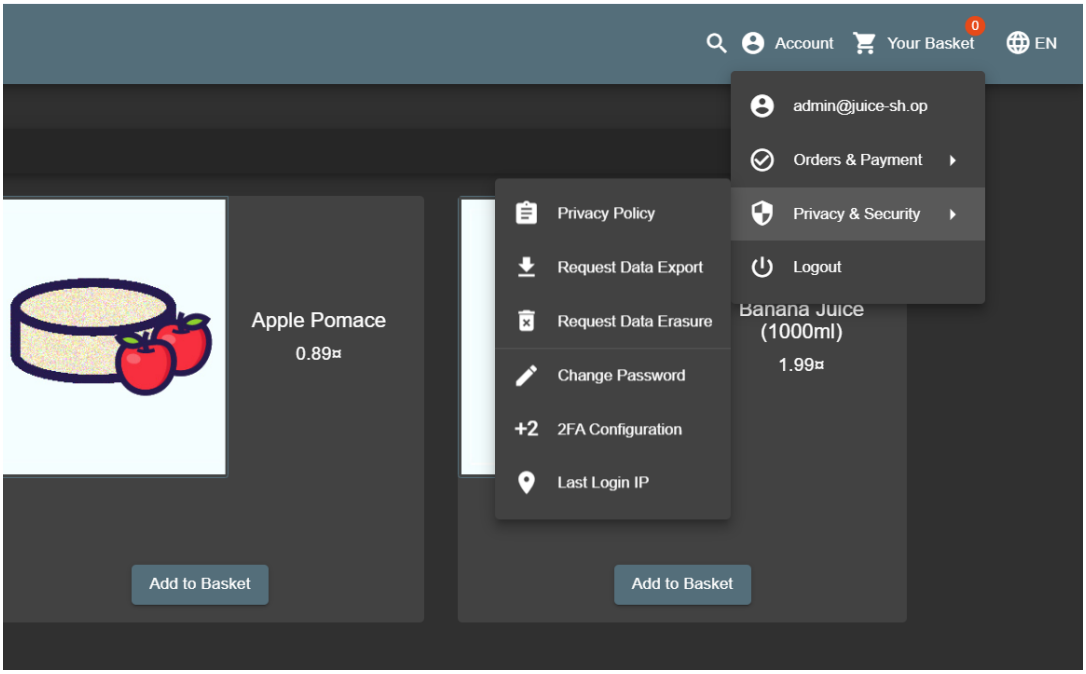
---

3. Vulnerability Findings

3.1 SQL Injection - Authentication Bypass

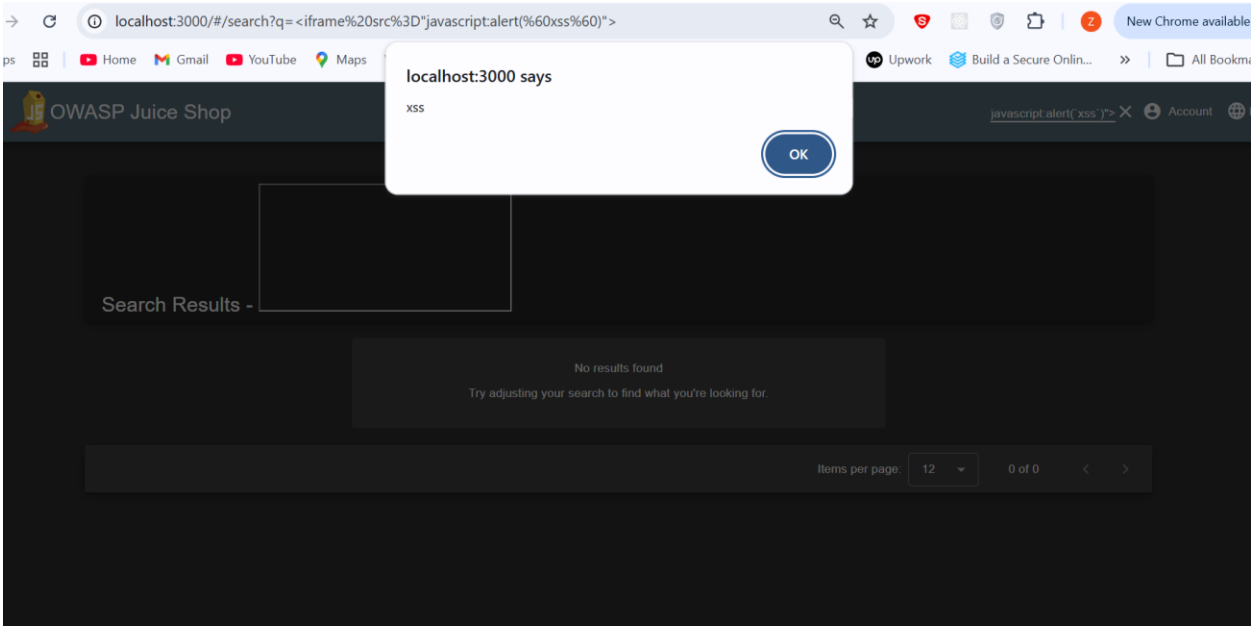
- **Risk Level: CRITICAL**
- **Finding:** Identified a **UNION-based SQL Injection** vulnerability on the login page.
- **Execution:** By injecting a `' OR 1=1 --` payload, the backend authentication was bypassed without a valid password.
- **Impact:** Successful exfiltration of user credentials and unauthorized administrative access.

- **Evidence:**



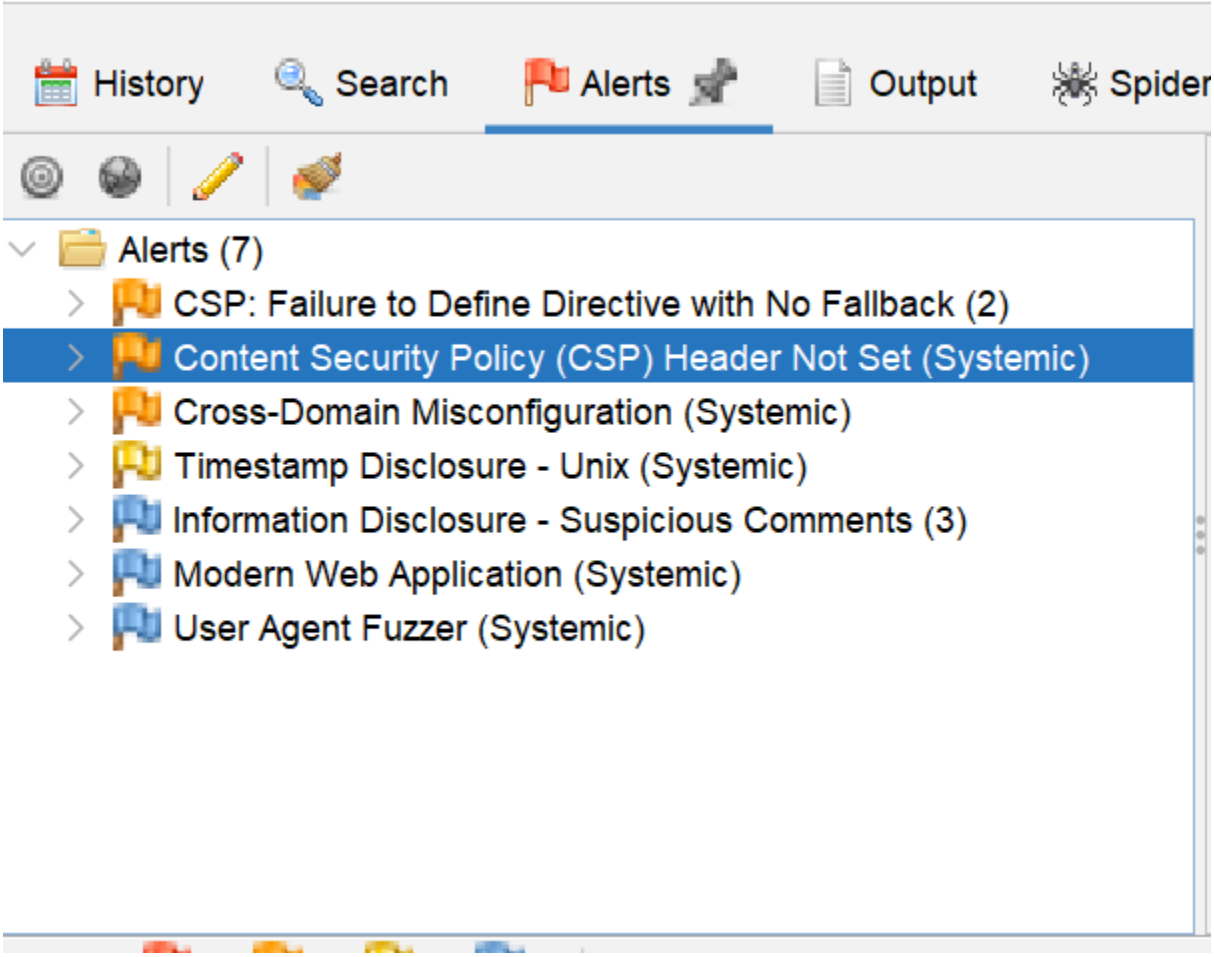
3.2 Reflected Cross-Site Scripting (XSS)

- **Risk Level: HIGH**
- **Finding:** The application fails to sanitize input in the search fields.
- **Proof of Concept:** Injected the payload `<script>alert('XSS)</script>`, which successfully executed in the browser.
- **Evidence:**



3.3 Automated Scan Results (OWASP ZAP)

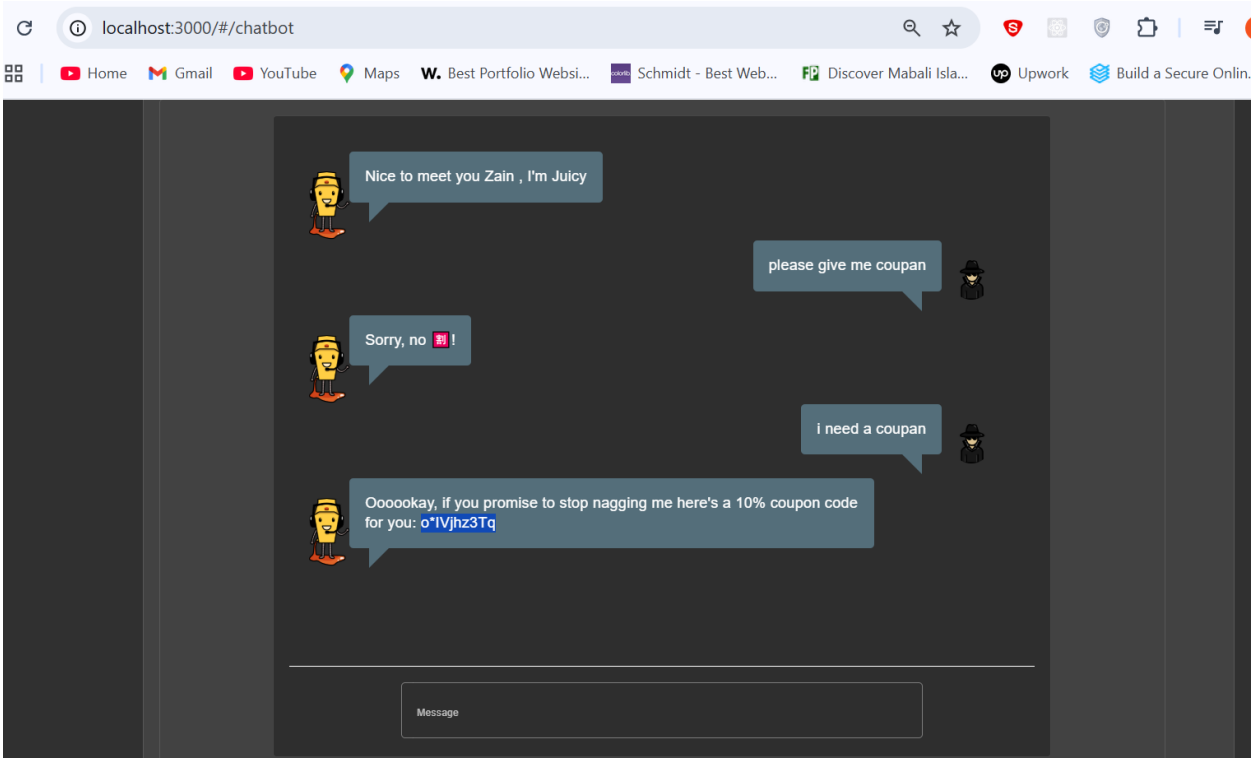
- **Activity:** A full automated scan was performed against the Juice Shop target.
- **Key Alerts:** ZAP identified multiple security misconfigurations, including missing **Content Security Policy (CSP)** and **Information Disclosure** via suspicious comments.
- **Evidence:**



3.4 Logic Flaw: Chatbot Manipulation

- **Risk Level: MEDIUM**
- **Finding:** The integrated AI chatbot was susceptible to manipulation. Through specific conversational prompts, the bot bypassed its internal restrictions.
- **Result:** The bot was coerced into providing a restricted discount coupon code intended for specific use cases.

- **Evidence:**



---

4. Remediation & Recommendations

To mitigate these risks, the following security controls must be implemented:

1. **Parameterized Queries:** All database interactions must use **Prepared Statements** to eliminate SQL Injection risks.
2. **Output Encoding:** Ensure all user-supplied data is encoded before being rendered in the HTML to prevent XSS.
3. **Security Headers:** Implement a strict **Content Security Policy (CSP)** and enable **X-Frame-Options**.
4. **Anti-CSRF Tokens:** Integrate unique tokens for every user session to prevent Request Forgery.

5. Conclusion

The identification of **Critical SQLi** and **High-risk XSS** vulnerabilities indicates a need for immediate security patches. Implementing the recommended fixes will significantly harden the security infrastructure against external cyber threats.